

Module 1

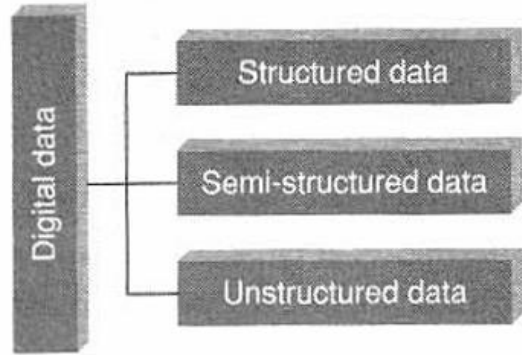
Classification of data, Characteristics, Evolution and definition of Big data, What is Big data, Why Big data, Traditional Business Intelligence Vs Big Data, Typical data warehouse and Hadoop environment.

Big Data Analytics: What is Big data Analytics, Classification of Analytics, Importance of Big Data Analytics, Technologies used in Big data Environments, Few Top Analytical Tools , NoSQL, Hadoop.

TB1: Ch 1: 1.1, Ch2: 2.1-2.5,2.7,2.9-2.11, Ch3: 3.2,3.5,3.8,3.12, Ch4: 4.1,4.2

Chapter 1

Classification of digital data



Structured data

- When data conforms to a predefined **schema/structure**, then it is a structured data.
- When dealing with **structured data**, we need a **data model**—a blueprint that defines how business data is **stored, processed, and accessed**.
- In an **RDBMS (Relational Database Management System)**:
 - **Data is stored in tables** (with rows and columns).
 - **Each table follows a relational model**, meaning data is organized logically.

Steps in Table Design:

1 Define the table structure

- Decide **columns (fields)** and their **data types** (e.g., numbers, text, dates).

2 Set constraints (rules for data)

- UNIQUE** → No duplicate values.
- NOT NULL** → Column cannot be empty.
- Business rules** → Example: A price column should not be below 50.
- Allowed values** → Example: A department column can only store "CS", "IS", "MS".
- The tables in RDBMS can be related by using primary key and foreign key concepts.
- Example, in table 1.3, **employee table** is a **referencing table** and **department table** is a **referenced table**.

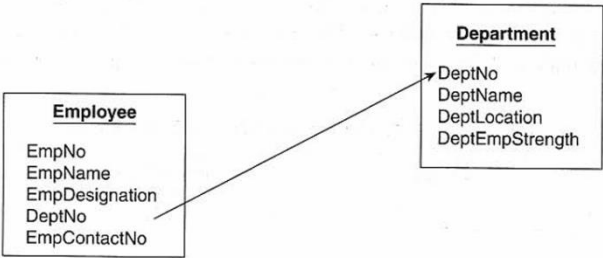


Figure 1.3 Relationship between “Employee” and “Department” tables.

Table 1.3 Sample records in the “Employee” table

EmpNo	EmpName	Designation	DeptNo	ContactNo
E101	Allen	Software Engineer	D1	0999999999
E102	Simon	Consultant	D1	0777777777

Table 1.1 A relation/table with rows and columns

Column 1	Column 2	Column 3	Column 4
Row 1			

Table 1.2 Schema of an “Employee” table in a RDBMS such as Oracle

Column Name	Data Type	Constraints
EmpNo	Varchar(10)	PRIMARY KEY
EmpName	Varchar(50)	
Designation	Varchar(25)	NOT NULL
DeptNo	Varchar(5)	
ContactNo	Varchar(10)	NOT NULL

Sources of structured data

- If your data is **highly structured** (organized in rows and columns), you can use a **Relational Database Management System (RDBMS)** like:
 - **Oracle, IBM DB2, Microsoft SQL Server** (popular enterprise databases)
 - **MySQL, PostgreSQL** (open-source databases)
 - **Teradata, Greenplum** (used for large-scale data storage)
- These databases store **transactional data** from daily business activities, such as:
 - ✓ Customer orders
 - ✓ Banking transactions
 - ✓ Employee records
- This type of data is managed by **On-Line Transaction Processing (OLTP) systems**, which handle **fast and frequent database operations** like inserting, updating, and deleting records.

1 Databases (RDBMS)

- Includes databases like **Oracle, DB2, Teradata, MySQL, PostgreSQL, etc.**
- Used for storing large-scale structured data with relational tables.
- Follows a predefined schema and supports **transactions** (ACID properties).

2 Spreadsheets

- Tools like **Excel or Google Sheets** store structured data in tabular format.
- Used for small-scale data storage and analysis.
- Lacks advanced features like indexing and transaction management.

3 OLTP (Online Transaction Processing) Systems

- Systems that handle real-time transactions (e.g., banking, e-commerce).
- Generates structured data continuously.
- Relies on RDBMS to manage transactions efficiently.

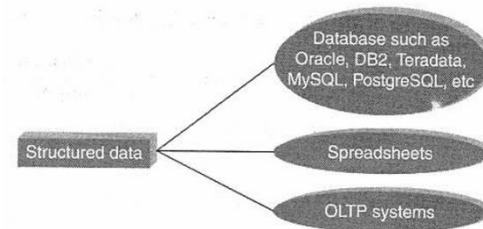


Figure 1.4 Sources of structured data.

Ease of working with structured data

- Structured data is easy to manage due to the following features:

1. Insert/Update/Delete – SQL operations (DML) simplify data input, storage, and processing.

2. Security – Encryption and tokenization ensure data protection and compliance.

- **Encryption:** Converts data into a coded format using encryption algorithms. Only authorized users with the correct decryption key can access the original data.
 - **Example:** A credit card number `1234-5678-9012-3456` is encrypted as `Xb4!z9Q@7f%Y2m#P`.
 - Even if a hacker steals the encrypted data, they cannot read it without the key.
- **Tokenization:** Replaces sensitive data with a random token, keeping the original data stored securely elsewhere.
 - **Example:** A customer's Social Security Number (SSN) `987-65-4321` is replaced with `A1B2C3D4E5`.
 - The real SSN is stored securely, and only authorized systems can retrieve it.

3. Indexing – Indexes speed up data retrieval at the cost of extra storage.

An index is a **data structure** that speeds up the data retrieval operations (primarily the SELECT DML statement) at the cost of **additional writes and storage space**, but the benefits that ensure in search operation are worth the additional writes and storage space.

4. Scalability: The storage and processing capabilities of the traditional RDBMS can be easily scaled up by increasing the horsepower of the data base server (increasing the primary and secondary or peripheral storage capacity, processing capacity of the processors etc.)

5. Transaction processing:

- Supports **ACID** properties:
- ✓ **Atomicity** – Transactions are fully completed or not at all.
- ✓ **Consistency** – Data remains accurate and valid.
- ✓ **Isolation** – Transactions execute independently.
- ✓ **Durability** – Changes are permanent once committed.

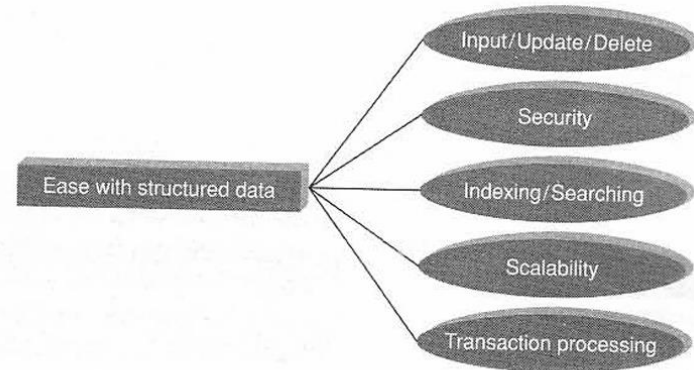


Figure 1.5 Ease of working with structured data.

Semi-structured data

- It is a self-describing structure.

1. No Fixed Data Model

- Semi-structured data **does not follow the strict table format** of relational databases (RDBMS).
- **Example:** Instead of storing customer details in a fixed table format, a JSON document may have flexible fields for each customer.

2. Uses Tags for Organization

- Instead of rows and columns, semi-structured data uses **tags** to define different parts of the data.

```
xml  
  
<book>  
  <title>Data Science Basics</title>  
  <author>John Doe</author>  
  <price>25.99</price>  
</book>
```

XML - Extensible Markup Language

Here, <title>, <author>, and <price> are **tags** that help define the data.

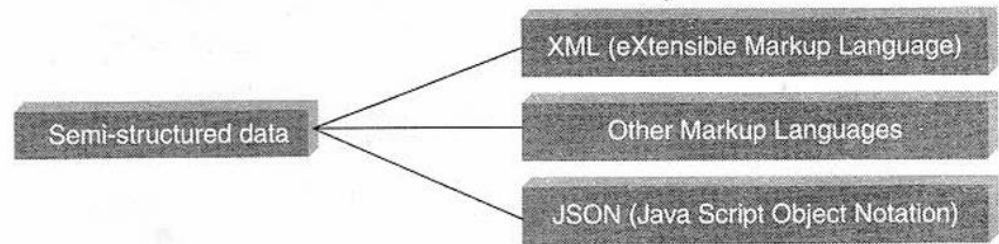


Figure 1.7 Sources of semi-structured data.

3. Tags Create Hierarchies

- Tags not only store data but also create a structure (like parent-child relationships).

xml

```
<company>
  <department name="IT">
    <employee>
      <name>Alice</name>
      <position>Software Engineer</position>
    </employee>
    <employee>
      <name>Bob</name>
      <position>Data Analyst</position>
    </employee>
  </department>

  <department name="HR">
    <employee>
      <name>Charlie</name>
      <position>Recruiter</position>
    </employee>
  </department>
</company>
```

json

```
{
  "store": {
    "electronics": {
      "items": [
        {"name": "Laptop", "brand": "Dell", "price": 1000},
        {"name": "Smartphone", "brand": "Samsung", "price": 800}
      ]
    },
    "furniture": {
      "items": [
        {"name": "Chair", "material": "Wood", "price": 50},
        {"name": "Table", "material": "Metal", "price": 120}
      ]
    }
  }
}
```

4. No Separation Between Data and Schema

- Unlike RDBMS, where schema (structure) is predefined, in semi-structured data, **structure is flexible** and defined along with the data.

Example: Two customer records can have different fields:

```
json
```

```
{"name": "Bob", "email": "bob@email.com"}  
{"name": "Sara", "phone": "123-456-7890"}
```

5. Same Class, Different Attributes

- In relational databases, all rows in a table must have the same columns, but in semi-structured data, this is not necessary.

Example: Two product records may have different attributes:

```
json
```

```
{"product": "Laptop", "RAM": "16GB", "Storage": "512GB SSD"}  
{"product": "Smartphone", "Camera": "48MP", "Battery": "4000mAh"}
```

Unstructured Data -> Data that lacks a predefined format

Table 1.4 Few examples of disparate unstructured data

Twitter message	Feeling miffed ☹. Victim of twishing.
Facebook post	LOL. C ya. BFN
Log files	127.0.0.1 - frank [10/Oct/2000:13:55:36 -0700] "GET /apache_pb.gif HTTP/1.0" 200 2326 "http://www.example.com/start.html" "Mozilla/4.08 [en] (Win98; I; Nav)"
Email	Hey Joan, possible to send across the first cut on the Hadoop chapter by Friday EOD or maybe we can meet up over a cup of coffee. Best regards, Tom

Unstructured data lacks a fixed format and is not organized into predefined models like tables. It consists of raw data that does not fit neatly into traditional databases and requires additional processing for meaningful analysis.

Advantages of Unstructured Data:

- ✓ Can store vast amounts of information from diverse sources.
- ✓ Contains rich insights for analytics (e.g., customer behavior analysis).

Disadvantages of Unstructured Data:

- ✗ Hard to store and manage in traditional databases.
- ✗ Complex processing is needed to extract valuable insights.

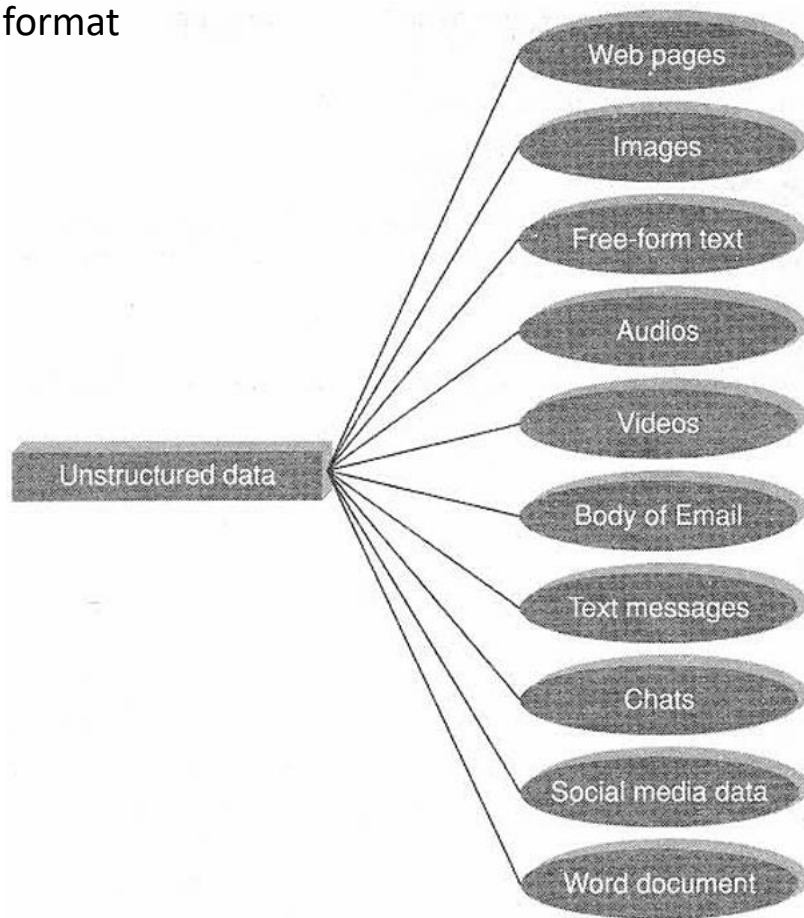
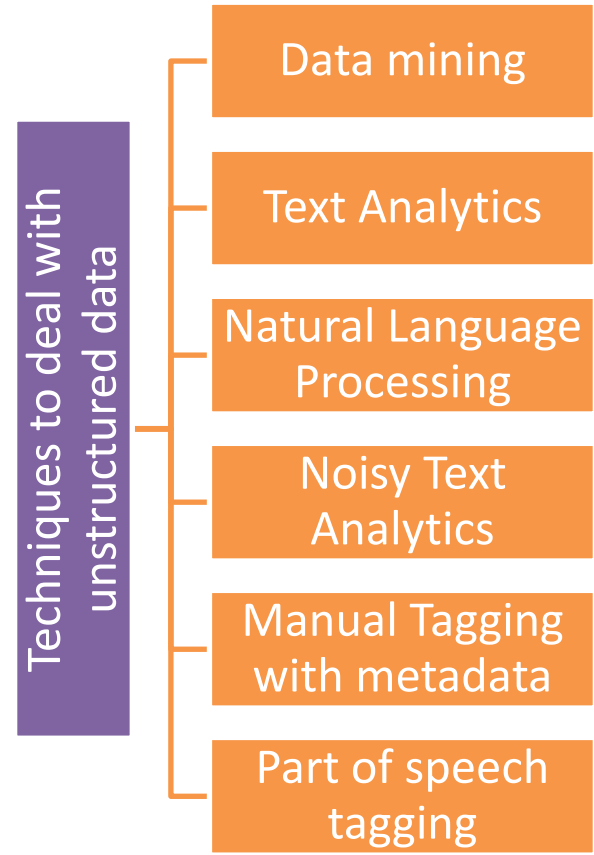


Figure 1.8 Sources of unstructured data.

How to deal with unstructured data?

- Unstructured data makes up about **80%** of the total data generated in businesses today. This includes text documents, images, videos, emails, social media posts, and more. Unlike structured data (which fits neatly into tables and databases), unstructured data **does not follow a fixed format**, making it difficult to manage and analyze.
- Since unstructured data is growing rapidly, companies must find ways to **process, analyze, and extract useful information** from it. Below are the techniques used to deal with unstructured data.



1) Data Mining (Finding patterns in large datasets)

Data mining is the process of **discovering patterns and relationships** in large amounts of data using AI, machine learning, and statistics. It helps in making predictions and finding useful insights.

◆ Popular Data Mining Techniques:

- **Association Rule Mining (Market Basket Analysis)**
 - Finds relationships between items in a dataset.
 - Example: If a person buys **bread**, they are likely to also buy **butter** or **cheese**.
 - Used by supermarkets to suggest products.
- **Regression Analysis**
 - Identifies the relationship between variables.
 - Example: If **advertising spending increases**, will **product sales increase**?
- **Collaborative Filtering (Recommendation Systems)**
 - Predicts user preferences based on the behavior of similar users.
 - Example: **Netflix recommends movies** based on what similar users watched.

Regression analysis: It helps to predict the relationship between two variables. The variable whose value needs to be predicted is called the dependent variable and the variables which are used to predict the value are referred to as the independent variables.

PICTURE THIS...

You are interested in purchasing real estate. You have been looking at a few good sites. You have come to the conclusion that cost of the real estate depends on the location (outskirts or prime locale), the amenities provided by the

builder (joggers track, senior citizen zone, gymnasium, swimming pools, etc.), the built up area, etc. The cost of the real estate is the dependent variable and the location, amenities, built-up area are called the independent variables.

2) Text Analytics (Text Mining)

- Since text is **unstructured**, analyzing it requires special techniques. **Text mining** extracts meaningful patterns from text data.

Uses of Text Mining:

- **Text Categorization** – Sorting emails as spam or not spam.
- **Text Clustering** – Grouping similar news articles together.
- **Sentiment Analysis** – Checking if product reviews are **positive or negative**.

3) Natural Language Processing (NLP)

- NLP helps computers **understand human language**. This is useful in chatbots, voice assistants, and translation tools.

Examples of NLP:

- **Google Translate** (Language translation).
- **Siri/Alexa** (Voice recognition).
- **Customer support chatbots** that understand and reply to queries.

4) Noisy Text Analytics

- Some unstructured text data contains **errors and inconsistencies** (such as typos, abbreviations, and informal language). Noisy text analytics helps in **cleaning and processing** such data.

Examples of Noisy Text:

- "u" instead of "you" (common in text messages).
- "Pls send d doc" instead of "Please send the document".
- Social media posts full of emojis and slang.

5. Manual Tagging with Metadata

- Metadata is **extra information** added to unstructured data to help in organizing and searching it.

Example:

- Adding **tags** to photos (e.g., "beach", "vacation", "friends") helps in searching them later.

6) Part-of-Speech (POS) Tagging

- This process **identifies the grammatical role** of each word in a sentence (noun, verb, adjective, etc.).

Example:

- In the sentence "**The cat runs fast**",
 - "Cat" is a **noun**.
 - "Runs" is a **verb**.
 - "Fast" is an **adjective**.
- This is useful for **language translation, speech recognition, and AI chatbots**.

7) Unstructured Information Management Architecture (UIMA)

- UIMA is an **IBM open-source platform** for processing unstructured data in real-time. It helps businesses extract useful **insights from documents, emails, social media, and customer feedback**.

Solve this

Place Me in the Basket

Structured	Unstructured	Semi-Structured

Following words are to be placed in the relevant basket:

Email
MS Access
Images
Database
Chat conversations

Relations/Tables
Facebook
Videos
MS Excel
XML

Structured	Unstructured	Semi-Structured
MS Access	Email	XML
Database	Images	
Relations/Tables	Chat conversations	
MS Excel	Facebook	
	Videos	

Match the Following

Column A	Column B
NLP	Content analytics
Text analytics	Text messages
UIMA	Chats
Noisy unstructured data	Text mining
Data mining	Comprehend human or natural language input
Noisy unstructured data	Uses methods at the intersection of statistics, AI, machine learning & DBs
IBM	UIMA

Answer:

Column A	Column B
NLP	Comprehend human or natural language input
Text analytics	Text mining
UIMA	Content analytics
Noisy unstructured data	Text messages
Data mining	Uses methods at the intersection of statistics, AI, machine learning & DBs
Noisy unstructured data	Chats
IBM	UIMA

Evolution of Big data

Table 2.1 The evolution of big data

	Data Generation and Storage	Data Utilization	Data Driven
Complex and Unstructured			Structured data, unstructured data, multimedia data
Complex and Relational		Relational databases: Data-intensive applications	
Primitive and Structured	Mainframes: Basic data storage		
	1970s and before	Relational (1980s and 1990s)	2000s and beyond

Definition of big data

Anything beyond the human and technical infrastructure needed to support storage, processing and analysis.

Today's BIG may be tomorrow's NORMAL

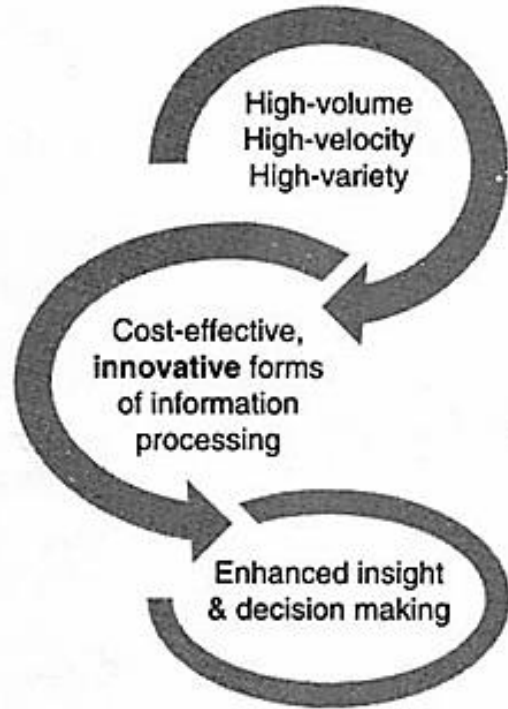
Terabytes or petabytes or zettabytes of data

I think it is about 3V's

- High Volume
- High Velocity
- High Variety

Big data is high volume, high velocity and high variety information assets that demand cost effective, innovative forms of information processing for enhanced insight and decision making.

The 3Vs concept was proposed by the Gartner analyst Doug Laney in a 2001 MetaGroup research publication, titled, *3D Data Management: Controlling Data Volume, Variety and Velocity*.



- **Large, Fast, and Varied Data** – Big Data includes huge amounts of structured, semi-structured, and unstructured data that need to be stored, processed, and analyzed quickly.
- **Advanced Processing Techniques** – New technologies and methods are required to handle and analyze this data efficiently.
- **Better Decisions and Business Value** – Analyzing Big Data helps organizations gain insights, make smarter decisions, and stay competitive.

Figure 2.3 Definition of big data – Gartner.

What is Big Data? (-> Volume)

1. What is Volume in Big Data?

- Big Data means a massive amount of data growing every day. We have moved from small data sizes like **kilobytes (KB)** and **megabytes (MB)** to **petabytes (PB)**, **exabytes (EB)**, and even **yottabytes (YB)**!

2. Where Does This Data Come From?

- Big Data comes from many sources, both inside and outside organizations.

➤ Internal Data Sources (Inside an Organization)

- **Storage Systems:** Databases like MySQL, PostgreSQL, MongoDB, and file systems.
- **Archives:** Scanned documents, student records, patient health records, etc.

➤ External Data Sources (Outside an Organization)

- **Public Websites:** Wikipedia, weather reports, government records, etc.
- **Both Internal + External Data**
- **Sensors:** Data from car sensors, smart meters, air conditioners, etc.
- **Logs:** Event logs, audit logs, website click data, etc.
- **Social Media:** Facebook, Twitter, YouTube, Instagram, etc.
- **Business Apps:** ERP (Enterprise Resource Planning), CRM (Customer Relationship Management), Google Docs, etc.
- **Media Files:** Audio, video, images, podcasts.
- **Documents:** PDFs, Word files, Excel sheets, PowerPoint presentations.

Bits → Bytes → Kilobytes → Megabytes → Gigabytes → Terabytes
→ Petabytes → Exabytes → Zettabytes → Yottabytes

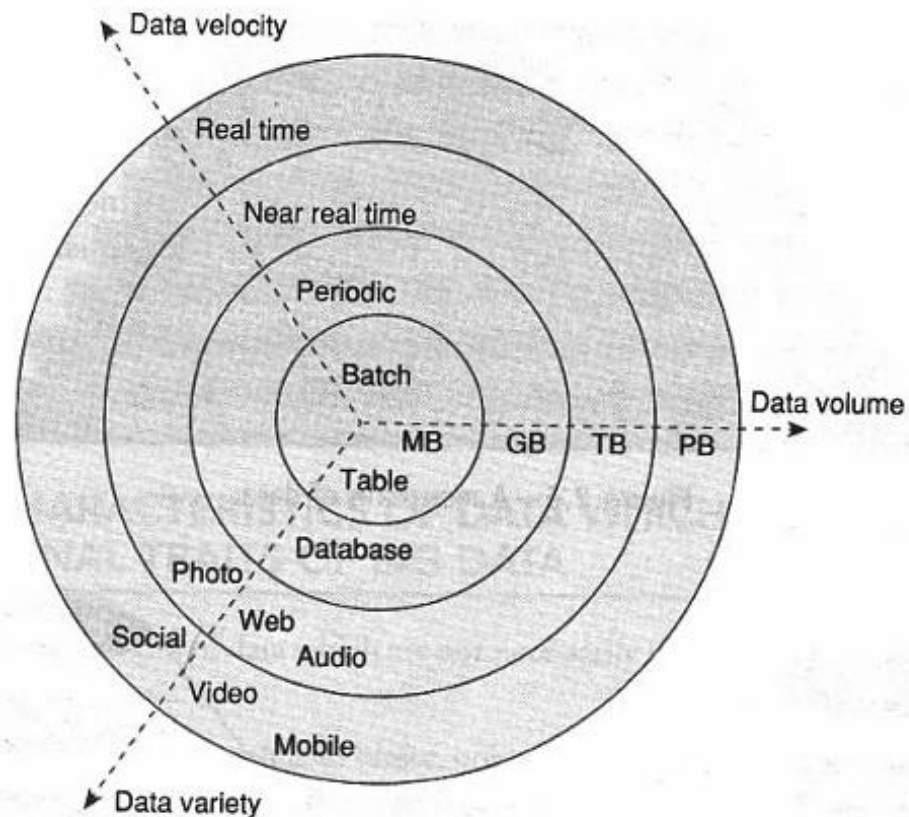


Figure 2.5 Data: Big in volume, variety, and velocity.

Table 2.2 Growth of data

Bits	0 or 1
Bytes	8 bits
Kilobytes	1024 bytes
Megabytes	1024^2 bytes
Gigabytes	1024^3 bytes
Terabytes	1024^4 bytes
Petabytes	1024^5 bytes
Exabytes	1024^6 bytes
Zettabytes	1024^7 bytes
Yottabytes	1024^8 bytes

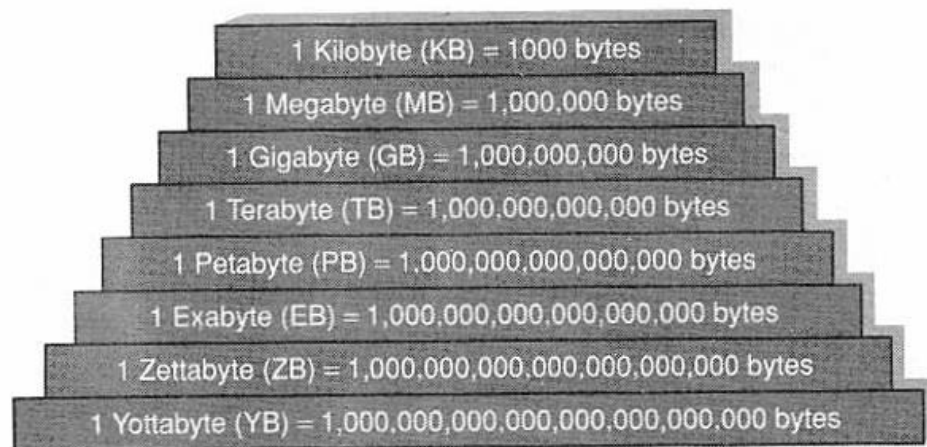


Figure 2.6 A mountain of data.

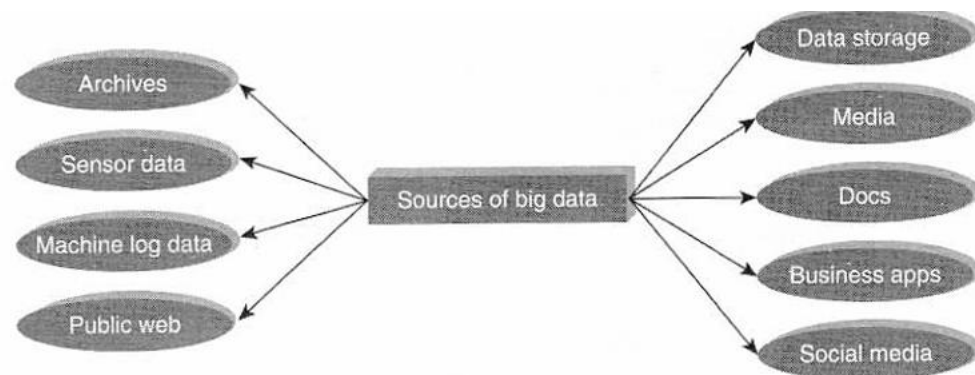


Figure 2.7 Sources of big data.

What is Big Data? (-> Velocity)

- **Velocity** in Big Data refers to **how fast data is generated, processed, and used**.
- We have moved from **slow batch processing** to **real-time processing** where data is handled instantly.

Types of Processing (with Examples)

❖ Batch Processing (Slow & Periodic)

- **Data is collected over time and processed in bulk at scheduled intervals.**
- **Example:** Monthly payroll processing—salary calculations happen once a month.

❖ Near Real-Time Processing (Faster, but Slight Delay)

- **Data is processed almost immediately but with a small delay.**
 - **Example:** Bank fraud detection—transactions are checked for fraud within minutes.
- ### ❖ Real-Time Processing (Instant & Continuous)
- **Data is processed immediately as it is generated.**
 - **Example:** Stock market trading—share prices update instantly based on market changes.

What is Big Data? (-> Variety)

- **Variety** refers to different types of data that come from various sources. It is categorized into three types:
 - rigid as tables.
 - **Example:** HTML (used in web pages), XML (used for data storage and sharing).

Structured Data (Well-Organized & Easily Searchable)

- Stored in tables with rows and columns (like databases).
- **Example:** Bank transactions, student records, sales data in SQL databases.

Semi-Structured Data (Partially Organized)

- Has some structure but is not as

Unstructured Data (No Fixed Format & Hard to Organize)

- Comes in different formats and is difficult to store in tables.
- **Example:** Emails, videos, social media posts, PDFs, images, audio recordings.

Why Big Data?

Step 1: More Data 📊

- Big Data means collecting a large amount of information from different sources.
- **Example:** An e-commerce company collects data about customer purchases, reviews, website visits, and product searches.

Step 2: More Accurate Analysis 🔍

- With more data, businesses can analyze trends and patterns more accurately.
- **Example:** The e-commerce company notices that customers buy more electronics during festive seasons.

Step 3: More Confidence in Decision-Making ✅

- Accurate data analysis helps in making better and more confident decisions.
- **Example:** Based on the trend, the company decides to offer discounts on electronics before the festival.

Step 4: Business Benefits & Efficiency 💡

- Using Big Data, businesses can improve efficiency, reduce costs, save time, and develop better products.
- **Example:** The e-commerce company sells more electronics, increases profits, and improves customer satisfaction with well-timed offers.

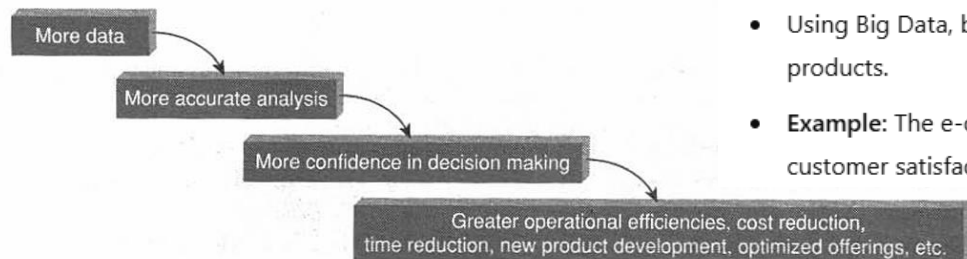


Figure 2.8 Why big data?

Traditional Business Intelligence vs Big Data

- **Business Intelligence (BI)** and **Big Data** are **two approaches used for analyzing data**, but they differ significantly in terms of data storage, processing, and analysis techniques.

1) Data Storage Architecture

- **Traditional BI:** **Stores all data in a central server** (typically a relational database). Scaling requires **upgrading the server** (vertical scaling).
- **Big Data:** Uses a **distributed file system**, where **data is spread across multiple servers**. Scaling is done by **adding more servers** (horizontal scaling).

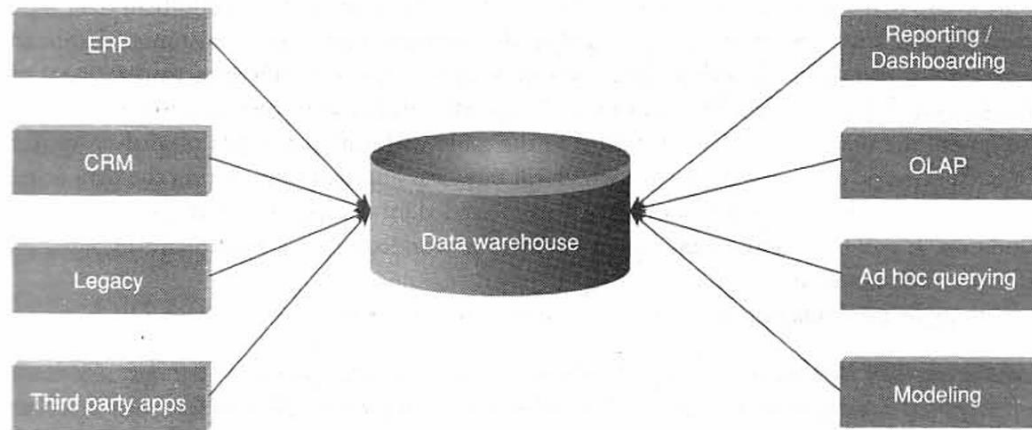
2) Data Processing Mode

- **Traditional BI:** Mostly works in **offline mode, analyzing historical data**.
- **Big Data:** Supports both **real-time (instant insights)** and **offline analysis**.

3) Data Type and Processing Approach

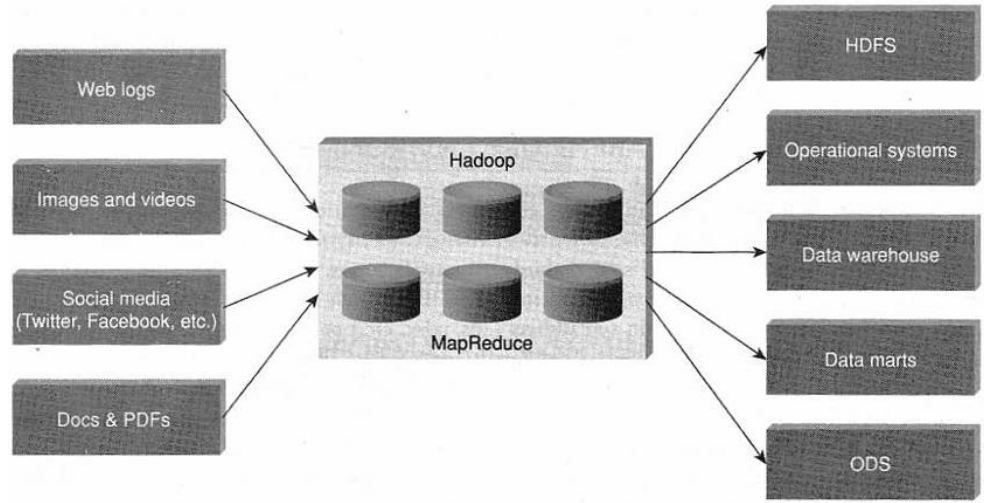
- **Traditional BI:** Works with **structured data** (tables, relational databases). Processing requires moving data to analytical functions.
- **Big Data:** Handles **structured, semi-structured, and unstructured data** (text, images, videos, logs, etc.). Here, processing functions are moved to where data is stored (closer to data).

A typical Data warehouse environment



- Operational or day-to-day business **data is gathered** from Enterprise Resource Planning(ERP) systems, Customer Relationship Management(CRM), legacy systems, and several third party applications.
- The **data from these sources may differ in format** (data could have been housed in any RDBMS such as Oracle, MS SQL Server, DB2, MySQL, and Teradata, and so on or in spreadsheet (.xls, .xlsx, etc) or .csv or .txt).
- **Data** may come **from** data sources located in the **same geography or different geographies**.
- This **data is** then **integrated, cleaned up, transformed, and standardized** through the process of **extraction, transformation and Loading(ETL)**.
- The **transformed data** is then **loaded into** the **enterprise data warehouse** or **data marts**.
- A host of market leading **business intelligence and analytics tools** are then used to **enable decision making** from the use of ad-hoc queries, SQL, Enterprise dashboard, data mining etc.

A Typical Hadoop Environment



- A **Data Warehouse** focuses on **structured data** from internal business sources like ERP, CRM, and databases.
- However, **Hadoop** can handle **both structured and unstructured data** from a wide range of sources, including web logs, social media, images, videos, and documents.

Example: Social Media Analytics in Hadoop

- Imagine a **social media company (like Twitter or Facebook)** that collects massive amounts of data daily:
- **Text data** (user posts, comments, hashtags)
- **Images and videos** (uploaded by users)
- **Clickstream data** (user interactions, likes, shares)

This data is:

- **Stored in HDFS** (Hadoop Distributed File System), as it supports large-scale unstructured data storage.
- **Processed using Hadoop tools** like **MapReduce, Hive, and Spark** to analyze trends, detect spam, and recommend content.
- **Insights can be sent to Data Warehouses** for structured reporting, or used directly for AI-driven recommendations.

Chapter 2

Big Data Analytics

What is big data analytics?

- **Big Data Analytics** refers to using technology to process and analyze large and complex datasets to gain valuable business insights.
1. **Technology-Enabled Analytics**
 - Tools like **IBM, Tableau, SAS, R Analytics, and Statistica** help **analyze Big Data**.
 - These tools **process data and create visualizations** for better understanding.
 2. **Deeper Business Insights**
 - Helps businesses **understand customer preferences, improve sales (cross-selling, up-selling), and optimize vendor relationships**.
 - **Example:** An online store tracks a customer's purchase history (favorite brand and color) and suggests similar products via email.
 3. **Competitive Advantage**
 - **Faster and better decisions** give companies an edge over competitors.
 - Helps in **trend prediction** and **customer targeting**.
 4. **Collaboration Between IT, Business, and Data Scientists**
 - **IT teams manage infrastructure, business users define needs, and data scientists extract insights**.
 5. **Handling Large & Diverse Datasets**
 - Big Data involves **massive volume (terabytes to zettabytes) and variety (text, images, videos, logs, etc.)**.
 - Traditional systems struggle with this scale, **requiring distributed computing**.
 6. **Moving Code to Data**
 - Instead of moving large data to programs, **small programs (a few KBs) are sent to process data where it resides (Hadoop-style processing)**.

Classification of Analytics

- There are basically two schools of thought:
 - a) Those that classify analytics into **basic, operationalized, advanced and monetized**.
 - b) Those that classify analytics into **analytics 1.0, analytics 2.0, and analytics 3.0**

First School of thought

1) Basic Analytics:

- **Analyzing past data to generate simple reports and basic visualizations.**
- **Example:** Reporting on last month's sales or creating charts to show trends in customer behavior.

2) Operationalized Analytics:

- **Using analytics directly within business operations to improve processes.**
- **Example:** A retail company uses data to recommend products to customers in real time during their shopping experience.

3) Advanced Analytics:

- Predicting future trends and outcomes using complex models.

4) Monetized Analytics:

- Using data to generate direct revenue for the business.
- **Example:** Google uses **monetized analytics** to generate revenue through targeted advertising. It analyzes user behavior—search history, website visits, and online interactions—to predict and display relevant ads. Businesses pay Google for ad placements based on clicks and impressions (Pay-Per-Click model). By leveraging data insights, Google ensures higher engagement, directly converting analytics into revenue.

Second school of thought

Analytics 1.0	Analytics 2.0	Analytics 3.0
Era: mid 1950s to 2009	2005 to 2012	2012 to present
Descriptive statistics (report on events, occurrences, etc. of the past)	Descriptive statistics + predictive statistics (use data from the past to make predictions for the future)	Descriptive + predictive + prescriptive statistics (use data from the past to make prophecies for the future and at the same time make recommendations to leverage the situation to one's advantage)
Key questions asked: What happened? Why did it happen?	Key questions asked: What will happen? Why will it happen?	Key questions asked: What will happen? When will it happen? Why will it happen? What should be the action taken to take advantage of what will happen?
Data from legacy systems, ERP, CRM, and 3rd party applications.	Big data	A blend of big data and data from legacy systems, ERP, CRM, and 3 rd party applications.
Small and structured data sources. Data stored in enterprise data warehouses or data marts.	Big data is being taken up seriously. Data is mainly unstructured, arriving at a much higher pace. This fast flow of data entailed that the influx of big volume data had to be stored and processed rapidly, often on massive parallel servers running Hadoop.	A blend of big data and traditional analytics to yield insights and offerings with speed and impact.
Data was internally sourced.	Data was often externally sourced.	Data is both being internally and externally sourced.
Relational databases	Database appliances, Hadoop clusters, SQL to Hadoop environments, etc.	In memory analytics, in database processing, agile analytical methods, machine learning techniques, etc.

Why Big Data Analytics Important?

1. Reactive - Business Intelligence (BI):

- BI looks at **past data** to help you understand what happened.
- It provides key information (often via dashboards and reports) so the right person can make quick, informed decisions.
- A **sales dashboard that shows last quarter's performance**.

2. Reactive - Big Data Analytics:

- Similar to BI, this method examines large volumes of **historical data**.
- It helps analyze big datasets to identify trends and patterns after the fact, even though the data is static.
- **Analyzing millions of customer transactions to find buying patterns**.

3. Proactive - Analytics:

- This approach uses tools like **data mining, predictive modeling, and statistical analysis to forecast future trends**.
- It supports **decision making for what might happen next**, although it typically deals with data sets that are not massive.
- **Predicting future sales trends based on past data and market research**.

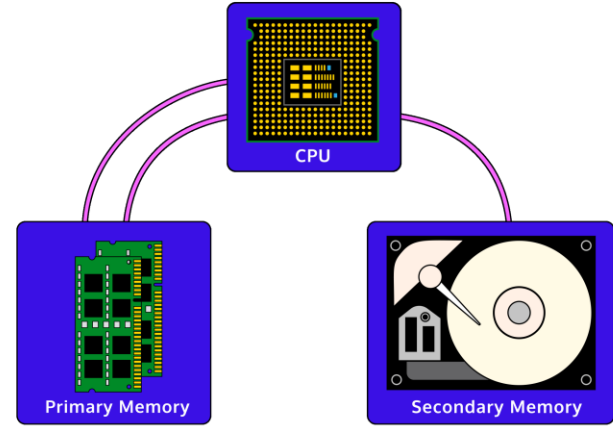
4. Proactive - Big Data Analytics:

- This is the next level, where the same **proactive techniques are applied to extremely large datasets** (terabytes or even petabytes).
- It **filters out the noise from huge volumes of data quickly**, providing **rapid insights to solve complex problems**.
- **Real-time fraud detection by analyzing streams of transaction data across the globe**.

Terminologies used in Big Data Environment

In-Memory Analytics:-

- Accessing data from a hard disk is slow. The more data needed, the slower it gets.
- A common solution is to **pre-process and store frequently used data** in advance, so only a small portion needs to be fetched.
- However, this requires **predicting what data will be needed**. If different data is required, the process must be repeated.
- **In-memory analytics** solves this problem by **storing relevant data in RAM (Random Access Memory) instead of a hard disk**.
- Since **RAM** is much **faster**, **data access** becomes **quicker**, leading to **faster insights**, **quicker deployment**.



Terminologies used in Big Data Environment

In-Database Processing

Online Transaction Processing (OLTP)

- In-database processing (or in-database analytics) means performing complex computations directly within the database instead of exporting data to separate analytical programs.
- **Data Fusion** – It integrates data warehouses with analytical systems.
- **ETL Process** – Data from OLTP systems is cleaned and stored in an Enterprise Data Warehouse (EDW) or data marts.
- **Traditional Method** – Data is usually exported to analytical programs for processing.
- **In-Database Processing Advantage** – The database itself performs computations, eliminating the need for exporting data, which saves time.
- Leading database vendors are offering this feature to large businesses.

Terminologies used in Big Data Environment

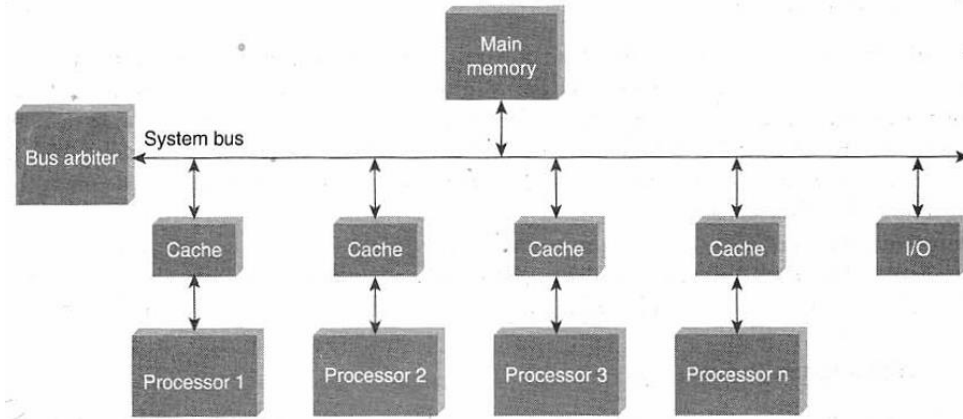


Figure 3.9 Symmetric Multiprocessor System.

Symmetric Multiprocessing (SMP)

- In **Symmetric Multiprocessing (SMP)**, multiple processors share a **single main memory** and work together.
- **Each processor** can access **all input/output (I/O) devices** and is managed by **one operating system**.
- These systems are called **tightly coupled multiprocessors** because the processors are closely connected.
- Each processor has its own **cache memory** (a small, fast memory) to store frequently used data.
- All **processors communicate** through a **system bus**, which **helps them exchange information efficiently**.

Terminologies used in Big Data Environment

Massively Parallel Processor

- **Massive Parallel Processing (MPP)** means **multiple processors work together** on the **same program** in **parallel**.
- Each processor has its **own operating system and memory**.
- They **communicate using messages**.
- MPP is **harder to program** because **tasks must be divided properly** for communication.

Difference SMP & MPP:

- **MPP:** Each processor has its own OS and memory (loosely coupled).
- **SMP:** All processors share the same OS and memory (tightly coupled).

Terminologies used in Big Data Environment

Difference between parallel and distributed systems

Parallel System	Distributed System
Tightly coupled system.	Loosely coupled system with multiple independent machines.
Multiple processors work together to process queries .	Each machine runs its own applications and serves its own users .
The user does not interact with individual processors. the system handles parallelism internally.	Data is distributed across multiple machines .
Processors either share a common memory or communicate via message passing .	Queries may require accessing multiple machines to retrieve data .
Think of it like a team working together in the same room. Multiple processors (or computers) work side by side, sharing memory or talking to each other (through message passing) to process a query. The user doesn't see any of this teamwork; it all happens behind the scenes. The system is "tightly coupled," meaning the processors are closely connected and cooperate directly.	Imagine several independent offices that each have their own staff and projects. In distributed systems, each machine operates on its own and may serve different users. The data is split across these machines. So, when a query is made, the system might need to reach out to several machines to get all the information. This setup is "loosely coupled" because the machines are more independent of each other.

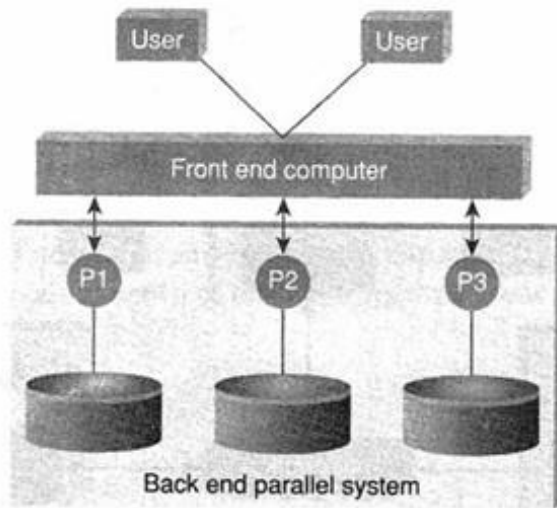


Figure 3.10 Parallel system.

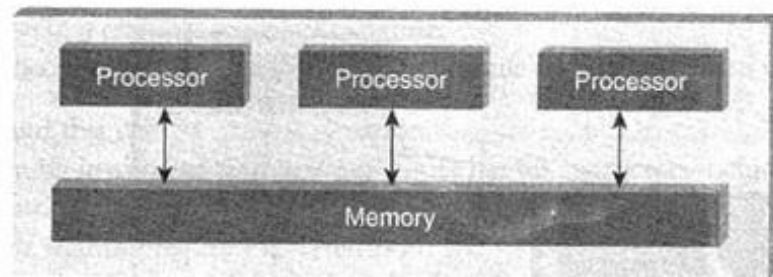


Figure 3.11 Parallel system.

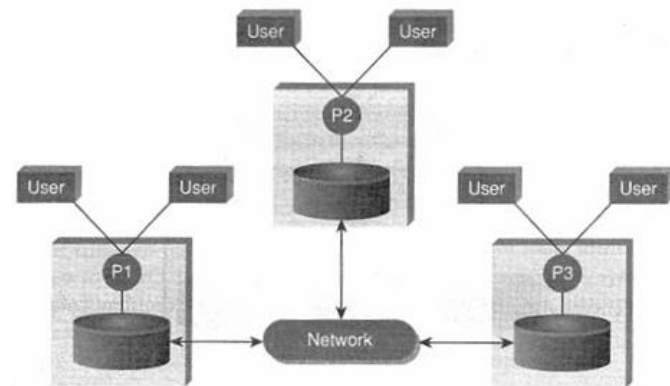


Figure 3.12 Distributed system.

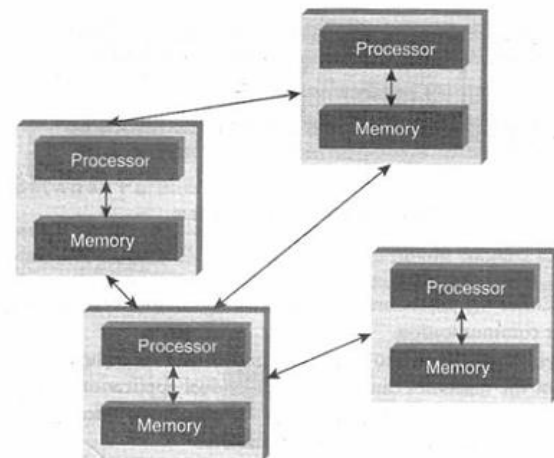


Figure 3.13 Distributed system.

Terminologies used in Big Data Environment

Shared Nothing Architecture

There are three **common architectures** used for **high transaction rate systems with multiple processors**:

1. **Shared Memory (SM)**: Multiple processors share a central memory.
2. **Shared Disk (SD)**: Each processor has its own memory but shares a common set of disks.
3. **Shared Nothing (SN)**: Each processor has its own memory and disk, with no shared components.

Advantages of Shared Nothing Architecture

- **Fault Isolation**: A failure in one node does not affect others, as each node operates independently.
- **Scalability**: Since no components are shared, more nodes can be added without performance bottlenecks related to synchronization.

CAP Theorem

The **CAP theorem** (also called Brewer's Theorem) states that in a **distributed system**, it is **impossible** to achieve all three of the following guarantees simultaneously:

- **Consistency (C)** – Every read fetches the latest write.
- **Availability (A)** – The system always responds to read/write requests.
- **Partition Tolerance (P)** – The system continues to function even if network failures occur.

Consistency	Availability	Partition Tolerance
• The system always gives the most recent and correct data. • Every user sees the same updated information at the same time. • Ex: Bank Transactions	• The system never refuses a request, even if it sometimes gives slightly outdated data. • It prioritizes uptime over accuracy. • Ex:- Even if some Google servers are down, your search results always appear.	• Even if parts of the system lose connection, it keeps working. • Some responses may be delayed, but the system does not completely shut down. • Ex: Sending of whatsapp message, which is in waiting mode, sends later when the network is restored.

Choosing Between Consistency and Availability

- **Choose Availability (AP) over Consistency** if slight delays in data updates are acceptable.
- **Choose Consistency (CP) over Availability** if accurate data is critical at all times.

Partition Tolerance (P) is **always required** in **distributed systems** because networks can fail or become slow due to:

- **Server crashes**
- **Network delays**
- **Data center outages**

Since distributed systems work across multiple nodes (servers), **they must keep functioning even if some parts of the network fail.**

Database Examples Based on CAP Theorem

- **AP (Availability + Partition Tolerance)** – DynamoDB, Cassandra
- **CP (Consistency + Partition Tolerance)** – HBase, MongoDB (configurable)
- **CA systems exist only in single-node (non-distributed) databases** like MySQL, where partition tolerance is not a concern.

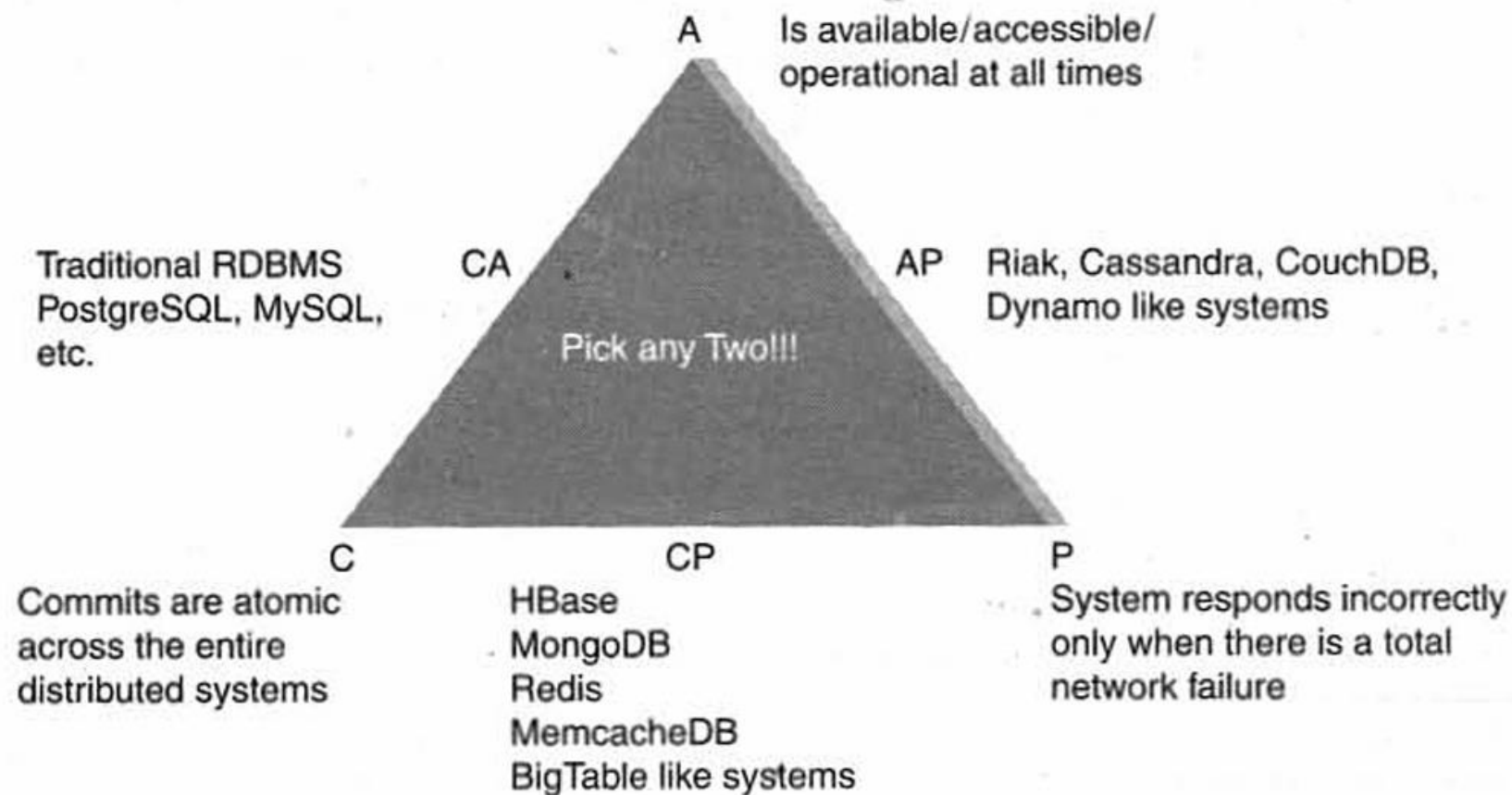
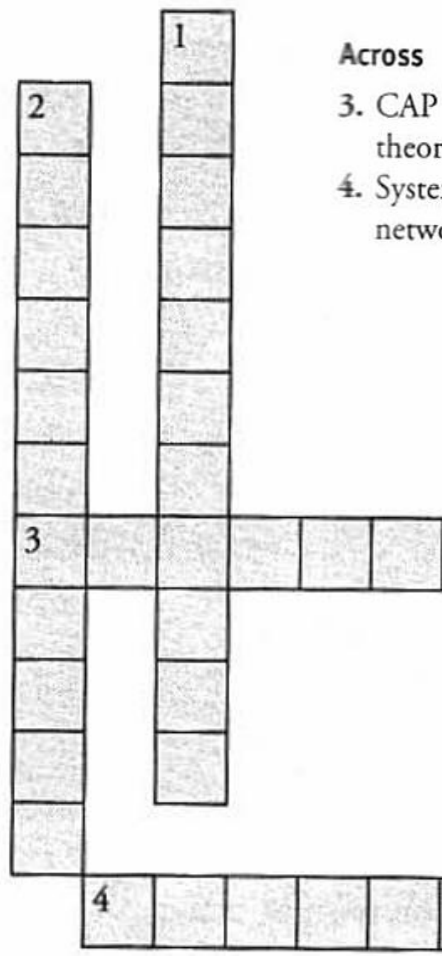


Figure 3.15 Databases and CAP.

Puzzle on CAP Theorem



Across

3. CAP theorem is also called as _____ theorem.
4. System will continue to function even when network partition occurs.

Down

1. Every read fetches the most recent write.
2. A non-failing node will return a reasonable response within a reasonable amount of time.

Solution:

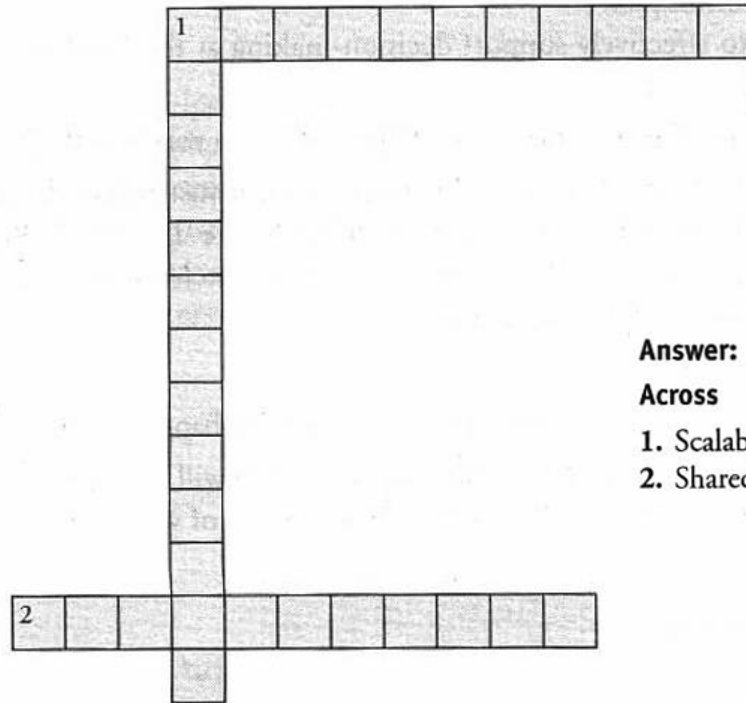
Across

3. Brewer
4. Partition Tolerant

Down

1. Consistency
2. Availability

2. Puzzle on Architecture



Answer:

Across

1. Scalability
2. Shared Disk

Down

1. Shared Memory

Across

1. _____ is an important advantage of shared nothing architecture.
2. In this architecture, multiple processors have their own private memory.

Down

1. In this architecture, central memory is shared by multiple processors.

NoSQL(Not only SQL)

Topics Covered

- ❖ Features of NoSQL databases
- ❖ Where is it used?
- ❖ What is it?
- ❖ Types of NoSQL databases.
- ❖ Why NoSQL?
- ❖ Advantages of NoSQL
- ❖ What we miss with NoSQL?
- ❖ Use of NoSQL in Industry.
- ❖ NoSQL Vendors
- ❖ SQL versus NoSQL
- ❖ NewSQL and its characteristics
- ❖ Comparison of SQL, NoSQL, and NewSQL

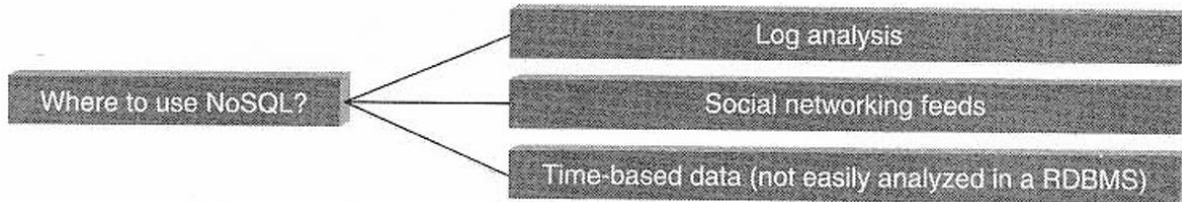
Few features of NoSQL database

- ❑ **Schema-less** – No fixed table structure; data can be stored flexibly.
- ❑ **Distributed Architecture** – Data is spread across multiple servers for reliability.
- ❑ **Scalability** – Easily handles large amounts of data by adding more servers.
- ❑ **High Performance** – Fast read/write operations due to simple data models.
- ❑ **Variety of Data Models** – Supports key-value, document, column-family, and graph models.
- ❑ **Handles Unstructured Data** – Works well with JSON, XML, and other formats.
- ❑ **No Complex Joins** – Queries are simpler compared to relational databases.
- ❑ **Eventual Consistency** – Focuses on availability and partition tolerance over strict consistency.

Where is NoSQL used?

NoSQL databases are commonly used for **big data** and **real-time web applications** because they **handle large** and **fast-changing data efficiently**.

- They are useful for **storing log data**, which can later be analyzed.
- They are also used for **storing social media data** and other types of data that **do not fit well** in traditional relational databases (RDBMS).



Log Data

- It's a **record of events or activities** happening in a system, application, or network.
- Automatically **generated and stored** for **tracking, analysis, and troubleshooting**.
- Examples of Log data: server logs, Error Logs, Security Logs, Transaction Logs.

What is NoSQL?

NoSQL (Not Only SQL) databases are **non-relational**, **open-source**, and **distributed**. They are popular because they can **handle large amounts of data efficiently** and **scale horizontally (adding more servers)**.

Important features of NoSQL Databases:

- **Non-Relational** – They do not follow the traditional relational model. Instead, they store data as **key-value pairs, documents, columns, or graphs**.
- **Distributed** – Data is spread across **multiple servers (nodes)** in a **cluster**, making them more efficient and fault-tolerant.
- **No ACID Support** – Unlike SQL databases, NoSQL databases **do not strictly follow ACID (Atomicity, Consistency, Isolation, Durability)**. Instead, they follow **CAP theorem** (Consistency, Availability, Partition Tolerance), often prioritizing availability over consistency.
- **Schema Flexibility** – NoSQL **does not require** a **fixed table structure**. It allows **storing data without defining a strict schema**, making it more flexible.

Types of NoSQL databases

1. Key-Value Databases

- Stores data as a **big hash table** (which makes use of hash functions for storing and retrieval of data) with **unique keys and values**.
- **Each key is linked to a value**, which can be text, JSON, or other data types.
- **Example Databases:** DynamoDB, Redis, Riak

Key	Value
Name	Micky
Age	19
Country	Germany

2. Document Databases

- Stores data as **documents** (JSON, BSON, or XML format).
- Documents are grouped into **collections** (like tables in SQL).
- **Example Databases:** MongoDB, CouchDB, Couchbase, MarkLogic

JSON (JavaScript Object Notation)

BSON (Binary JSON)

```
{  
  "Book Name": "Fundamentals of Business Analytics",  
  "Publisher": "Wiley India",  
  "Year of Publication": "2011"  
}
```

3. Column-Family Databases

- Instead of storing data **row by row** (like SQL), it **groups related columns together**. This **makes queries faster when retrieving data** from specific columns.

Example: Student Data in a Traditional SQL Table (Row-Based Storage)

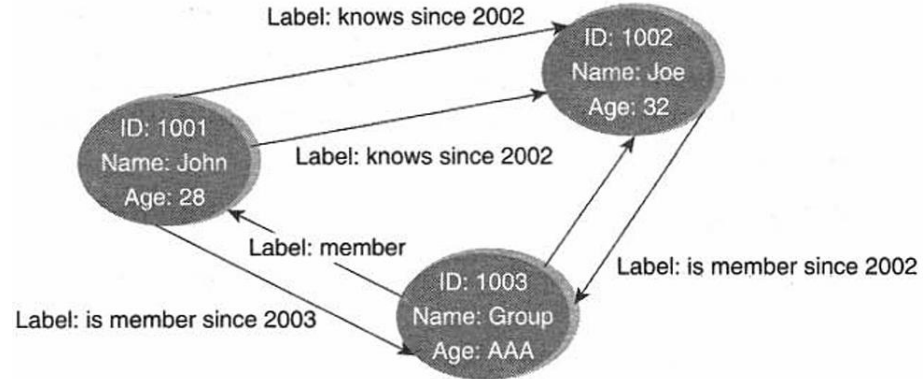
Student ID	Name	Course	Marks	Grade
101	Alice	Math	90	A
102	Bob	Science	85	B
103	Charlie	History	88	B+

◆ Column Family: Student_Info

Student ID	Name	Course
101	Alice	Math
102	Bob	Science
103	Charlie	History

◆ Column Family: Student_Grades

Student ID	Marks	Grade
101	90	A
102	85	B
103	88	B+



4. Graph Databases

- Stores data as **nodes (entities) and relationships** instead of tables.
- Best for applications where **relationships between data matter** (e.g., social networks).
- **Example Databases:** Neo4j, HyperGraphDB

Example (Graph Storage)

- **Nodes:** (Person: Alice), (Person: Bob)
- **Relationship:** (Alice - [FRIEND_OF] → Bob)

Why Use NoSQL?

- **Scalable Architecture** – Unlike traditional databases that rely on a single powerful server (**monolithic architecture**), NoSQL scales by **adding multiple servers** (scale-out architecture).
- **Handles Large & Varied Data** – Supports **structured, semi-structured, and unstructured** data, making it ideal for big data applications.
- **Flexible Schema** – NoSQL databases allow **storing data without a fixed structure**. This makes it **easy to modify data formats** without major changes, supporting **faster development**.
- **Auto-Sharding** – Data is **automatically split and distributed** across multiple servers. If a server fails, the system **replaces it without downtime**.
- **Replication for Reliability** – NoSQL databases support **data replication across multiple locations**, ensuring **high availability, fault tolerance, and quick recovery** from failures.

Advantages of NoSQL

1. Scalable and Fast

- Can **scale up or down** easily.
- Supports:
 - **Cluster Scale** – Can distribute data across 100+ servers.
 - **Performance Scale** – Handles 100,000+ read/write operations per second.
 - **Data Scale** – Can store **billions of documents**.

Example: Large-scale applications like Facebook or Amazon use NoSQL to handle massive amounts of data efficiently.

2. No Fixed Schema (Flexible Structure)

- Unlike relational databases, **NoSQL does not require a predefined structure.**
- **Different records (documents) in the same collection can have different key-value pairs.**

```
{ "_id": 101, "BookName": "Fundamentals of Business Analytics", "AuthorName": "Secma Acharya", "Publisher": "Wiley India" }
```

```
{ "_id": 102, "BookName": "Big Data and Analytics" }
```

Here, the second document has fewer fields than the first, which is **not possible in SQL databases**.

Advantages of NoSQL

3. Cost-Effective and Easy to Use

- NoSQL databases **reduce operational costs** while offering **high availability and fault tolerance**.
- They are **simpler to implement** and **maintain**.

4. Prioritizes Availability Over Strict Consistency

- NoSQL follows the **CAP Theorem**, balancing:
 - **Consistency** (data is the same everywhere).
 - **Availability** (data is always accessible).
 - **Partition Tolerance** (handles network failures).
- Most NoSQL databases **compromise strict consistency** but ensure **eventual consistency** (data will sync over time).

Example: In an online shopping app, product availability may be **temporarily inconsistent** across different locations, but it will eventually be updated correctly.

Advantages of NoSQL

5. Supports Data Replication and Partitioning

(a) Sharding (Data Partitioning)

- **Data is split across multiple servers** for **efficient storage and retrieval**.

Example: A search engine stores different categories (news, sports, tech) on separate servers to improve performance.

(b) Replication (Data Copying)

- **Multiple copies of data** are **stored across different servers and locations**.
- Ensures **high availability** and **fault tolerance** (if one server fails, data is still accessible).

Example: Google Cloud and AWS replicate user data across multiple data centers to prevent data loss.

What we miss with NoSQL?

- NoSQL databases solve **scalability issues** by allowing easy expansion (**scaling out**) and providing **flexible schema design** (no fixed structure). However, they **lack some important features** of traditional SQL databases.

Limitations of NoSQL (and How It Compensates)

- **No Support for Joins** – Unlike SQL, NoSQL does not allow complex **joins** between tables.
 - ❓ **Solution:** It allows **embedded documents** (like in MongoDB), where related data is stored together to avoid joins.
- **No ACID Transactions** – NoSQL does not fully support **Atomicity, Consistency, Isolation, and Durability (ACID)** like RDBMS.
 - ❓ **Solution:** NoSQL follows **CAP Theorem** (Consistency, Availability, Partition tolerance), meaning it focuses more on performance and availability.
- **No Standard SQL Interface** – NoSQL does not use **SQL** for querying data.
 - ❓ **Solution:** Databases like **MongoDB** and **Cassandra** have their own **query languages** (MongoDB Query Language & Cassandra Query Language - CQL).
- **Difficult Integration with SQL-based Applications** – Many applications are designed to work with SQL, making NoSQL integration harder.

Feature	SQL (RDBMS)	NoSQL (How It Compensates)
Scalability	Vertical scaling	Horizontal scaling (Scaling out)
Joins	Supported	Uses embedded documents instead
ACID Transactions	Fully supported	Follows CAP theorem
Query Language	Standard SQL	Uses custom query languages (e.g., CQL)
Integration	Easy with SQL apps	Limited support for SQL-based apps

Use of NoSQL in Industries

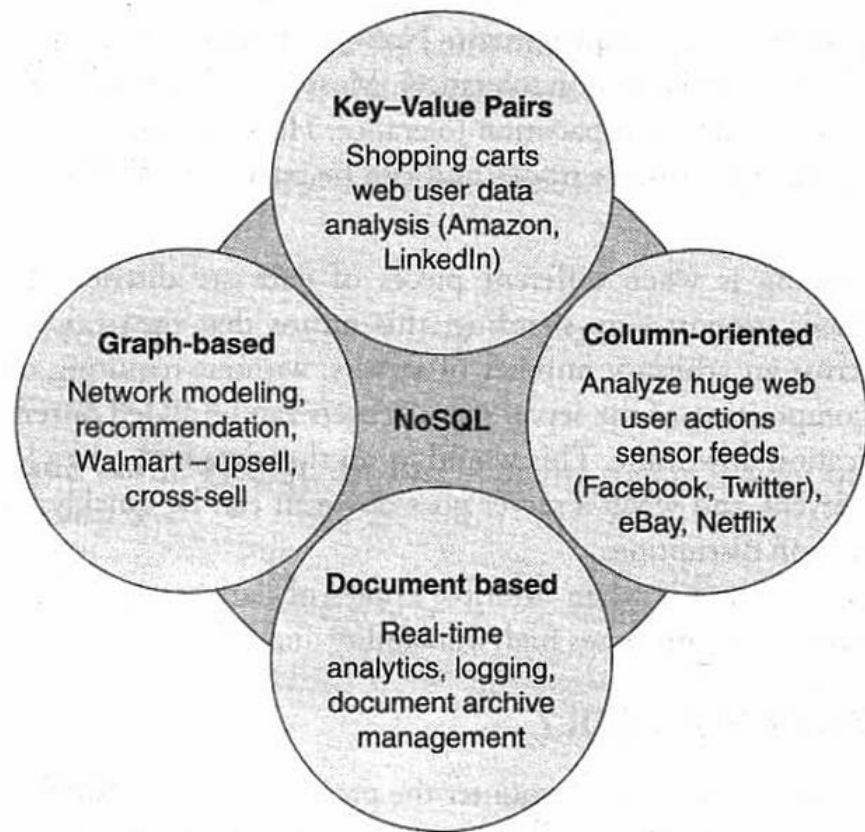


Figure 4.6 Use of NoSQL in industry.

NoSQL vendors

Table 4.2 Few popular NoSQL vendors

Company	Product	Most Widely Used by
Amazon	DynamoDB	LinkedIn, Mozilla
Facebook	Cassandra	Netflix, Twitter, eBay
Google	BigTable	Adobe Photoshop

SQL vs NoSQL

SQL	NoSQL
Relational database	Non-relational, distributed database
Relational model	Model-less approach
Pre-defined schema	Dynamic schema for unstructured data
Table based databases	Document-based or graph-based or wide column store or key-value pairs databases
Vertically scalable (by increasing system resources)	Horizontally scalable (by creating a cluster of commodity machines)
Uses SQL	Uses UnQL (Unstructured Query Language)
Not preferred for large datasets	Largely preferred for large datasets
Not a best fit for hierarchical data	Best fit for hierarchical storage as it follows the key-value pair of storing data similar to JSON (Java Script Object Notation)
Emphasis on ACID properties	Follows Brewer's CAP theorem
Excellent support from vendors	Relies heavily on community support
Supports complex querying and data keeping needs	Does not have good support for complex querying
Can be configured for strong consistency	Few support strong consistency (e.g., MongoDB), some others can be configured for eventual consistency (e.g., Cassandra)
Examples: Oracle, DB2, MySQL, MS SQL, PostgreSQL, etc.	Examples: MongoDB, HBase, Cassandra, Redis, Neo4j, CouchDB, Couchbase, Riak, etc.

NewSQL

NewSQL is a modern type of **relational database** that **combines** the best **features of both traditional SQL (RDBMS) and NoSQL databases**.

How is NewSQL different from SQL and NoSQL?

- **SQL (Traditional RDBMS): Supports ACID transactions** and **structured queries** but **struggles with scalability**.
- **NoSQL: Provides high scalability** and **better performance** for large-scale applications but **lacks strong ACID guarantees**.
- **NewSQL: Offers the scalability of NoSQL** while **maintaining the ACID properties of SQL**.

Why was NewSQL introduced?

We need a database that:

- **Scales like NoSQL** for handling large OLTP (Online Transaction Processing) workloads.
- **Ensures ACID properties** like SQL for **data consistency and reliability**.

Characteristics of NewSQL

1. SQL Interface for Applications

- NewSQL databases use **SQL** for queries, making it easy for developers to interact with them.

2. ACID Support for Transactions

- Unlike NoSQL, NewSQL **ensures ACID** for **reliable transactions**. (A – Atomicity, C – Consistency, I – Isolation, D – Durability)

3. High Performance Per Node

- It is designed to **process data faster per server (node)** compared to traditional relational databases.

4. Scalability with Shared-Nothing Architecture

- NewSQL follows a **"scale-out" model**, meaning more servers can be added as needed **without sharing memory or storage**.

5. Non-Locking Concurrency Control

- It uses an efficient **concurrency control** mechanism to allow **real-time reads and writes without slowing down performance**.

concurrency control ensures that multiple users can access and modify data **at the same time** without conflicts or errors.

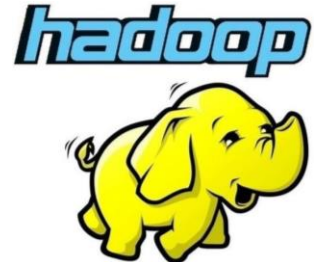
Comparative study of SQL, NoSQL, NewSQL

	SQL	NoSQL	NewSQL
Adherence to ACID properties	Yes	No	Yes
OLTP/OLAP	Yes	No	Yes
Schema rigidity	Yes	No	Maybe
Adherence to data model	Adherence to relational model		
Data Format Flexibility	No	Yes	Maybe
Scalability	Scale up Vertical Scaling	Scale out Horizontal Scaling	Scale out
Distributed Computing	Yes	Yes	Yes
Community Support	Huge	Growing	Slowly growing

Feature	SQL	NoSQL	NewSQL
Schema	Fixed	Schema-less/Dynamic	Fixed (but flexible in some cases)
Data Structure	Tables (Rows & Columns)	Key-Value, Document, Column-Family, Graph	Tables (Rows & Columns)
Flexibility	Low	High	Moderate
Best for	Structured data	Unstructured/Semi-structured data	Scalable structured data

Hadoop

- Hadoop is an **open-source project** of the **Apache foundation**. It is a **framework written in Java**, originally developed by **Doug Cutting in 2005** who named it after **his son's toy elephant**.
- He was working with **Yahoo** then. It was created to support distribution for **"Nutch", the text search engine**.
- Hadoop uses **Google's MapReduce** and **Google File System** technologies as its foundation. Hadoop is now a **core part of the computing infrastructure** for companies such as **Yahoo, Facebook, LinkedIn, Twitter, etc.**



Features of Hadoop

1. Handles All Types of Data

- Can process **structured, semi-structured, and unstructured** data.
- Uses **commodity hardware (cheap computers)**, making it cost-effective.

2. Shared-Nothing Architecture

- Each computer (node) works **independently without sharing memory or storage**.

3. Data Replication for Reliability

- Data is **copied across multiple computers**, so if one fails, another has the backup.

4. Hadoop is for high throughput rather than low latency. It is a batch operation handling massive quantities of data; therefore the response time is not immediate.

Why Can't Hadoop Replace RDBMS?

- *RDBMS is good for real-time transactions (OLTP). Hadoop is not fast enough for this.*
- *Hadoop is designed for large-scale batch processing, not quick queries.*
- *RDBMS supports structured data with ACID transactions, while Hadoop is better for unstructured data.*

5. If tasks depend on each other, Hadoop isn't the best choice. It is designed for parallel processing, not a sequential.

6. Works best with huge datasets, not small individual files.

Advantages of Hadoop

1. Stores Data Without a Fixed Format

- Hadoop's file system (**HDFS**) saves raw data **without needing a fixed structure (schema-less)**.
- Structure is added **only when needed for processing**.

2. Highly Scalable

- Can handle **huge datasets (terabytes to petabytes)** by distributing them across **many cheap servers**.

3. Cost-Effective

- Uses a **scale-out model**, meaning you can **add more low-cost servers** instead of expensive hardware.
- This reduces the **cost of storage and processing per terabyte**.

4. Fault-Tolerant

- **Automatically copies (replicates) data** across multiple servers.
- If one server **fails**, another has the backup, so **no data is lost**.

5. Works with Any Type of Data

- Supports **structured, semi-structured, and unstructured data** (e.g., emails, social media, logs).
- Used for **data mining, market analysis, recommendation systems**, etc.

6. Fast Processing

- Uses a **"move code to data"** method, meaning computations happen **where the data is stored** instead of transferring data to a processing system.
- This makes Hadoop **faster** than traditional database systems.

Versions of Hadoop

Hadoop has **two versions**:

- **Hadoop 1.0** – Basic version with **MapReduce** (Framework) for **data processing**.
- **Hadoop 2.0** – Improved version with **YARN** for better **resource management** and **flexibility**.

Hadoop 1.0

Two Main Components:

- **HDFS (Hadoop Distributed File System)** – **Stores data in its raw format without a fixed structure.**
- **MapReduce (Data Processing Framework)** – Uses two functions:
 - **Map**: Converts input data into key-value pairs.
 - **Reduce**: Processes and summarizes the data.

Limitations of Hadoop 1.0

- **Needed expertise in MapReduce & Java** – Hard to use for non-programmers.
- **Only supported batch processing** – No real-time processing.
- **MapReduce was compulsory** – Other data processing methods weren't supported, making it inefficient.

Hadoop 2.0

❑ Introduced YARN (Yet Another Resource Negotiator)

- Manages resources better than Hadoop 1.0.
- Allows multiple processing frameworks, not just MapReduce.

❑ More Flexibility

- Can run any application, not just MapReduce-based programs.

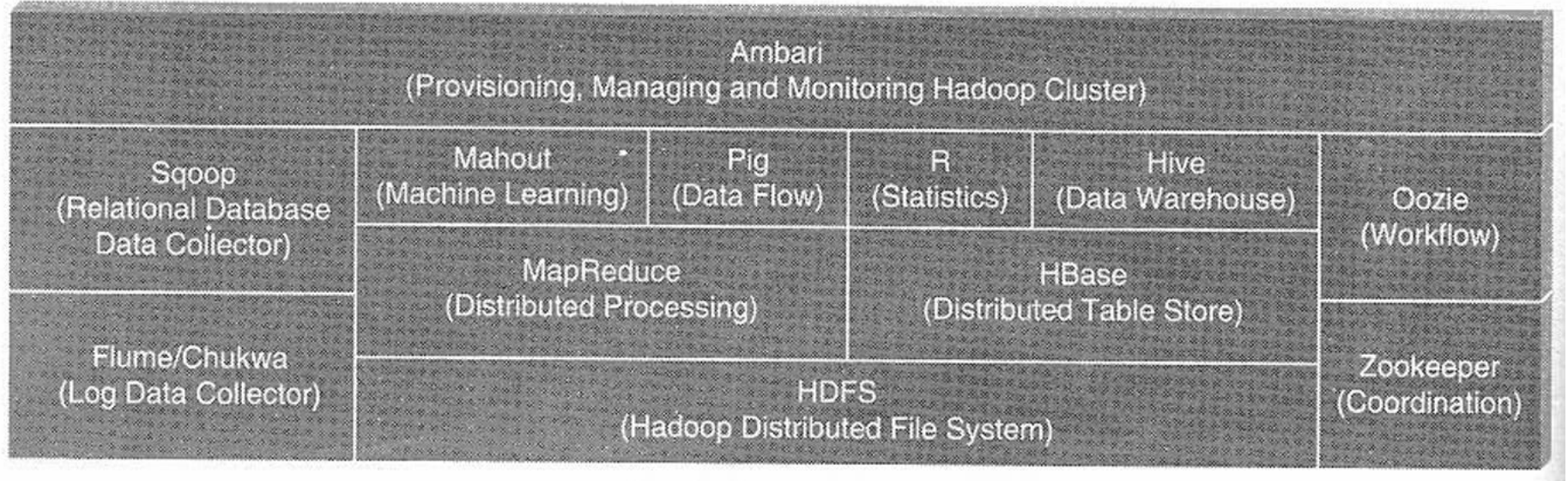
❑ Supports Real-Time Processing

- Unlike Hadoop 1.0, it can process data in real time, not just in batches.

❑ Better Performance & Scalability

- Handles more applications simultaneously without slowing down.

Overview of Hadoop Ecosystem



There are components available in the Hadoop ecosystem for data ingestion, processing and analysis.

Data Ingestion -> Data Processing -> Data Analysis

Data Ingestion

- Collecting raw data from different sources
- Sqoop & Flume

Data Processing

- Cleaning, transforming, and structuring data
- MapReduce & Spark

Data Analysis

- Finding insights and patterns in data
- Pig, Hive & Impala

HDFS

- HDFS is the **storage system of Hadoop** that splits large files and stores them across multiple computers.

Key Features:

- **Stores Big Data:** Can handle very large datasets. And it is used for **sequential access** of big data.
- **Distributed Storage:** Spreads data across many machines.
- **Fault-Tolerant:** If one machine fails, data is still available from other copies.
- **Inspired by Google File System (GFS).**

HBase

- HBase is a database built on top of HDFS, **designed for quick access to big data.**

Key Features:

- **Faster than HDFS:** Provides quick, **random access** to data.
- **NoSQL Database:** Stores data in a non-relational format.
- **Column-Oriented:** Data is stored in column-family data model.
- **Used for real-time applications** (e.g., Facebook, Twitter).
- **Based on Google BigTable.**

Think of **HDFS** as a **large library** 📖 where books (files) are stored **in bulk**—you must scan a lot to find something.

Think of **HBase** as a **catalog system** 📖 that quickly retrieves a book by its index.

- ✓ Use HDFS for storing huge amounts of data.
- ✓ Use HBase when you need fast lookups and updates.

Hadoop Ecosystem components for data ingestion

1) Sqoop (SQL to Hadoop)

- ❑ **Purpose:** Moves data between RDBMS (MySQL, Oracle, DB2) and Hadoop (HDFS, HBase, Hive).
- ❑ **Two Main Functions:**
 - **Import:** Transfers structured data from RDBMS to Hadoop. (RDBMS -> Hadoop)
 - **Export:** Moves processed data from Hadoop back to RDBMS. (Hadoop -> RDBMS)
- ❑ **Why use Sqoop?**
 - Works with SQL databases using connectors.
 - Automates data transfer with Oozie (a scheduler, which automates import and export tasks).
 - Helps populate tables (inserting or loading data into tables) in Hive & HBase for analysis.
- ❑ **Example:** Suppose you have a MySQL database with customer data. Sqoop can import it into HDFS for analytics and later export the processed data back to MySQL.

Hadoop Ecosystem components for data ingestion

2) Flume – Collects & Moves Streaming Data

- ❑ **Purpose:** Gathers **huge volumes of log or event data** and stores it in HDFS.
- ❑ Flume has been developed by **cloudera**(A company that provides Hadoop-based solutions)
- ❑ **How it Works?**
 - Collects **real-time logs** from **multiple sources (e.g., web servers)**.
 - **Sends the data** to a **storage location** (HDFS, HBase, or Solr).
 - The **default storage location** of **Flume** is **HDFS**.
- ❑ **Why use Flume?**
 - Handles **continuous data streams**.
 - Best for **log collection** from different machines.
- ❑ **Example:** Imagine a website with **millions of users**. Flume collects **click logs & user activities** from different servers and saves them in HDFS for analysis.

Tool	Purpose	Used for
Sqoop	Transfers structured data (RDBMS ↔ Hadoop)	Batch data movement (MySQL to HDFS, Hive, HBase, and vice versa)
Flume	Collects & moves real-time event logs to HDFS	Streaming data ingestion (web logs, social media feeds, server logs)

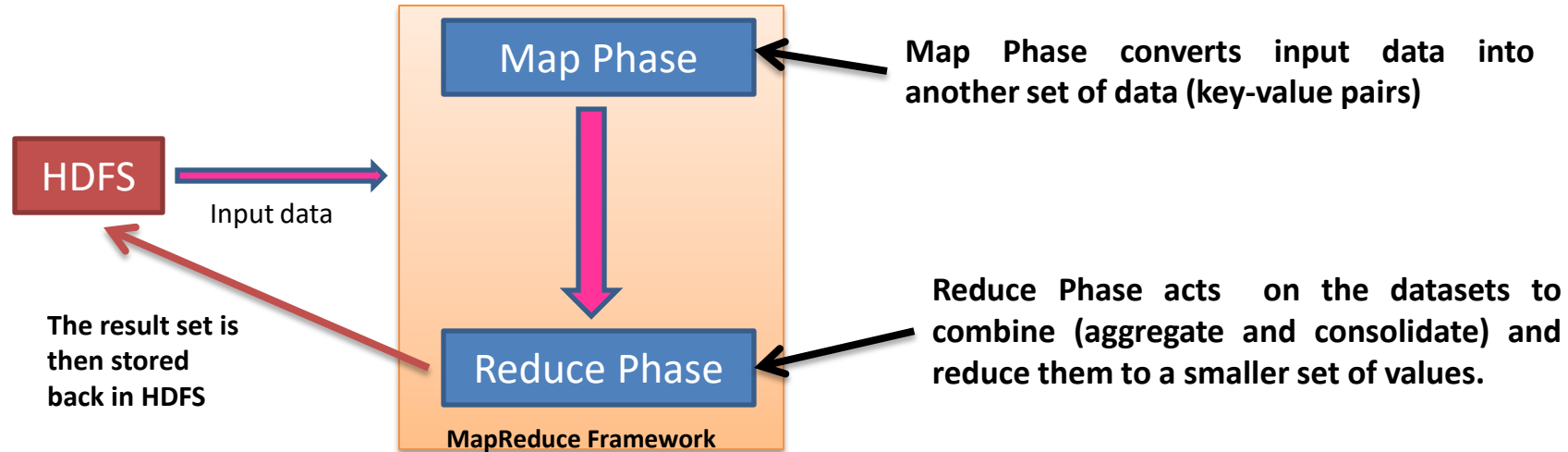
Sqoop = Batch Data Transfer

Flume = Real-time Data Ingestion

Hadoop Ecosystem components for Data Processing

1) MapReduce

- A **programming model** for **processing large datasets** in a **distributed and parallel way**.
- It is based on **Google MapReduce**.
- **Google released** a paper on **MapReduce programming** paradigm in **2004** and that became the **genesis of Hadoop processing model**.



- The MapReduce framework gets the input data from HDFS.
- There are **two main phases**: **Map phase** and the **Reduce phase**.
- The **map phase** converts the input data into another set of data (key-value pairs). This new intermediate dataset then serves as the input to the reduce phase.
- The **reduce phase** acts on the datasets to combine (aggregate and consolidate) and reduce them to a smaller set of tuples.
- The result is then stored back in HDFS.

Example: Suppose you have a large collection of documents and want to count the occurrence of each word.

Map Phase: Reads documents and converts words into key-value pairs (e.g., "Hadoop" → 1, "Big Data" → 1).

Reduce Phase: Aggregates these values to get the total count (e.g., "Hadoop" → 20, "Big Data" → 15).

2) Spark

- It is both a **programming model** as well as a **computing model**.
- It is an **open source big data processing framework**.
- It was originally **developed** in **2009** at **UC Berkeley's Amplab** and became an **open source project** in **2010**.
- It is **written** in **scala**.
- It provides **in-memory computing** for **hadoop**.
- In spark, **workloads execute in memory rather than on disk** owing to which it is much **faster** (10 to 100 times) than when the workload is executed on disk. However, if the **datasets are too large to fit into the available system memory**, it can **perform conventional disk-based processing**.
- It serves as a **potentially faster** and **more flexible alternative to mapreduce**.
- **Can run with or without Hadoop:**
 - Can use **HDFS** for storage.
 - Can work alongside MapReduce or as a standalone system.

Spark Components (Libraries):

- **Spark SQL** – Allows **querying big data** using **SQL**.
- **Spark Streaming** – **Processes real-time data** (e.g., live tweets, stock market updates).
- **MLlib (Machine Learning Library)** – Supports machine learning tasks like **classification and clustering**.
- **GraphX** – Helps in **working with graph data** (e.g., social network analysis).

Feature	Hadoop MapReduce	Spark
Speed	Slower (disk-based)	Faster (in-memory)
Processing Type	Batch Processing	Real-time + Batch
Flexibility	Less flexible	More flexible
Use Case	Large historical datasets (e.g., log analysis)	Fast computations (e.g., fraud detection, real-time analytics)

Hadoop Ecosystem components for Data Analysis

1) Pig

- Pig is a **high-level scripting tool** used for **processing large data sets** in **Hadoop**. It is an **alternative to writing complex MapReduce programs**.

❖ Components of Pig:

(a) **Pig Latin**: A **scripting language** similar to SQL, used for **writing data processing tasks**.

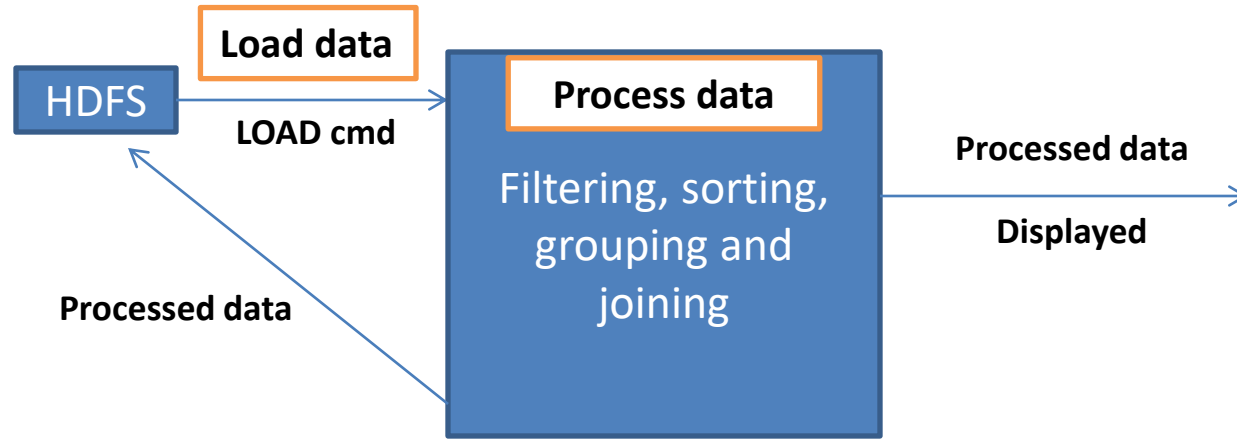
(b) **Pig Runtime**: The **execution environment** where **Pig scripts run**.

❖ How Pig Works?

- **Load Data** → Pig loads data from **HDFS** using the LOAD command.
- **Process Data** → Operations like filtering, sorting, grouping, and joining can be applied.
- **Store Data** → The processed data can be either displayed or stored back into HDFS.

Uses:

- Helps in **ETL (Extract, Transform, Load)** processes.
- Useful for developers who are not familiar with MapReduce.



2) Hive

- Hive is a **data warehouse software project** built on top of Hadoop.
- Three main tasks performed by Hive are **summarization, querying, and analysis**.
- It supports **queries** written in a language called **HQL or HiveQL** which is a declarative SQL-like language.
- It **converts** the **SQL-like queries into MapReduce jobs**(program that processes large amounts of data in a parallel and distributed manner) which are then **executed** on the **hadoop platform**.

Difference between Hive and RDBMS

Feature	Hive (Big Data Analysis)	RDBMS (Traditional Database)
Schema Handling	Schema on Read (applied when reading data, fast loading).	Schema on Write (checked during insertion, ensures data integrity).
Read & Write	Write once, read many times (best for analysis).	Read and write many times (best for frequent transactions).
Processing Type	Batch processing, suitable for OLAP (data analysis).	Real-time processing, suitable for OLTP (transactions like banking, shopping).
Data Type	Static (historical) data, slower queries.	Dynamic (real-time) data, faster queries.
Scalability & Ownership	Highly scalable, stores data in HDFS, does not "own" data.	Owns and manages data, but has limited scalability.
Computing Method	Uses parallel computing (processes data in multiple nodes).	Uses serial computing (processes tasks one by one).

Difference between Hive and Hbase

Feature	Hive (SQL Engine)	HBase (NoSQL Database)
Type	SQL-based batch processing tool.	Key-value NoSQL database for real-time data.
Use Case	Best for analyzing large datasets.	Best for real-time read/write operations.
Runs On	MapReduce & Hadoop.	HDFS (Hadoop Distributed File System).

Impala → Fast SQL engine for interactive queries (low latency).

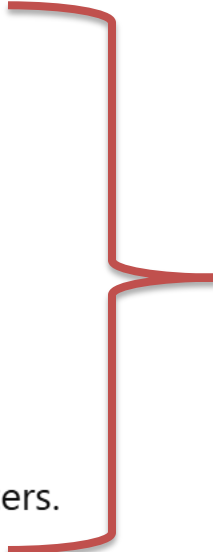
ZooKeeper → Manages and coordinates distributed applications.

Oozie → Schedules and manages Hadoop jobs.

Mahout → Machine learning and data mining library.

Chukwa → Collects and manages large system logs.

Ambari → Web-based tool for managing and monitoring Hadoop clusters.



**Other Hadoop
Ecosystem
components**

Hadoop Distributions

- Hadoop is an **open-source** project by **Apache** that helps store and process **big data** efficiently. Anyone can freely download and use it.

Core Components of Hadoop:

- 1 **Hadoop Common** → Shared libraries and utilities for all Hadoop modules.
- 2 **HDFS (Hadoop Distributed File System)** → Stores large data across multiple machines.
- 3 **YARN (Yet Another Resource Negotiator)** → Manages and allocates resources for tasks.
- 4 **MapReduce** → Processes large data in parallel across many machines.

Hadoop Distributions:

- Big companies like **IBM, AWS, Microsoft, Cloudera, Hortonworks, and Teradata** offer **customized versions of Hadoop** with added features to make it easier to use.

Why is Hadoop Important?

- **Scalable** → Can handle petabytes of data across many machines.
- **Efficient** → Distributes data and processing to speed up tasks.
- **Cost-effective** → Uses cheap hardware to manage big data.
- **In Short:** Hadoop helps companies **store, process, and analyze huge datasets** efficiently using multiple computers.

Feature	Hadoop (Big Data Processing)	SQL (RDBMS) (Traditional Databases)
Definition	An open-source framework for storing and processing big data across multiple machines.	A relational database system (RDBMS) for storing and managing structured data .
Storage	Uses HDFS (Hadoop Distributed File System) for distributed storage.	Uses databases like MySQL, PostgreSQL, Oracle, etc..
Data Type	Handles structured, semi-structured, and unstructured data.	Works best with structured data (tables, rows, and columns).
Processing Type	Uses batch processing (MapReduce) and parallel computing .	Uses real-time transactional processing .
Scalability	Highly scalable (handles petabytes of data).	Limited scalability (expensive to scale beyond a certain point).
Speed	Slower for small data , better for large-scale analysis.	Faster for small datasets and real-time queries.
Schema Handling	Schema on Read (applies schema when reading data).	Schema on Write (schema enforced when inserting data).
Best For	Big data analytics, batch processing, and unstructured data.	OLTP (Online Transaction Processing), real-time updates, and structured data management.

Integrated hadoop systems offered by leading market vendors

- An **Integrated Hadoop System** is a **ready-to-use** Hadoop setup provided by companies, making **big data processing easier, scalable, and efficient**.

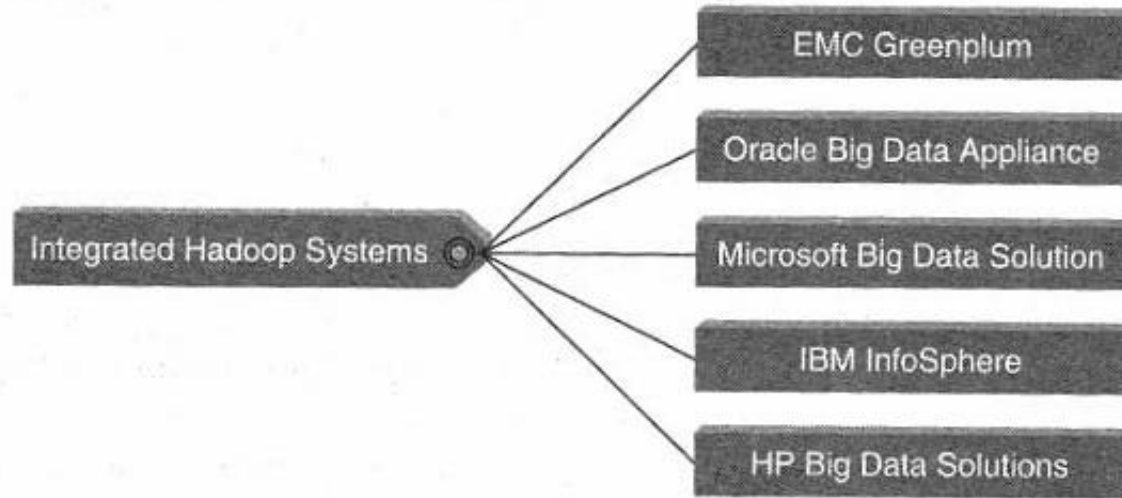


Figure 4.13 Integrated Hadoop systems.

Drawback of traditional Hadoop HDFS

In **traditional Hadoop HDFS**, the **NameNode** is the **central component** responsible for managing file metadata (such as file locations and structure). However, if the **NameNode fails**, the entire HDFS system **becomes inaccessible**, leading to a **single point of failure (SPOF)** issue.

Cloud based hadoop solutions

1) Amazon Web Services (AWS)

- Provides a **wide range** of cloud services to handle **big data**.
- Focuses on **lowering costs**, **scaling** to meet growing demands, and **speeding up innovation**.

2) Google Cloud Storage Connector for Hadoop

- Lets you **run MapReduce jobs** directly on data stored in **Google Cloud Storage**.
- **No need** to copy data to **HDFS** or local disks first.
- **Simplifies** Hadoop deployment and **reduces costs**.
- Offers **performance** similar to HDFS.
- Removes the **single point of failure** issue (the NameNode) found in traditional HDFS setups.

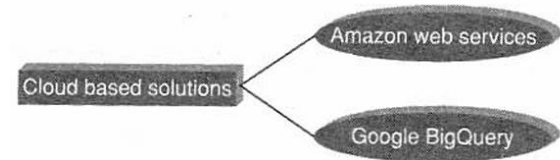


Figure 4.14 Cloud-based solutions.

Topics related to lab programs

HADOOP

- Hadoop is a framework (a system) that helps **store and process large amounts of data** across **multiple computers**.
- It is useful when **data is too big to be handled** by **a single computer**.
- It has 2 main components:

HDFS

- Hadoop Distributed File System(HDFS)
- Stores large files across multiple machine.
- To start HDFS services, in windows, use a command : start-dfs.cmd
- To stop HDFS services, in windows, use a command: stop-dfs.cmd

YARN

- Yet Another Resource Negotiator(YARN)
- Manages computing resources for processing data.
- To start YARN services, in windows, use a command: start-yarn.cmd
- To stop YARN services, in windows, use a command: stop-yarn.cmd

NameNode(Master for storage – HDFS)

- It acts as the **manager for HDFS**.
- It **keeps track** of **where files are stored across the cluster**.
- It does not **store** actual data but the **metadata**(file locations, block information, etc.)
- Example: Think of namenode as librarian who knows where the books(data blocks) are kept in a big library(HDFS). This librarian does not keep books but maintains a catalog.

DataNode (Worker for storage – HDFS)

- It **stores the actual data** in HDFS.
- Each **data node reports** to the **namenode about stored data**.
- If **one datanode fail**, **hadoop** automatically **retrieves the data** from **another copy**.
- Example: Think of datanodes as bookshelves in a library, where actual books (data) are stored.

ResourceManager (Master for Processing – YARN)

- It **manages cluster resources** like **CPU and memory**.
- It **decides which task should run on which machine**.
- It has two parts:
 - a) **scheduler**: allocates resources
 - b) **Applicationmanager**: Monitors tasks.

Nodemanager (worker for processing – YARN)

- **Runs** on **each worker machine**.
- It **executes tasks** assigned by the **resourcemanager**.
- **Monitors resource usage** and **reports back** to the **resourcemanager**.

start-all.cmd

- It will start all the services of hadoop and yarn.

hdfs namenode –format

- It is used to **initialize the Hadoop Distributed File System (HDFS) NameNode** by formatting its metadata storage

When is it used?

- **First-time setup** of an HDFS cluster.
- **Reinstalling HDFS**, when you need a fresh start.
- **Clearing all existing metadata.**